Pseudo Random Number Generators or PRNGs

COMP 445

Motivation

“The Monte Carlo method is a major industry and random numbers are its key resource”

Stephan Mertens

Lecture given at the International Summer School Modern Computational Science (August 16-28, 2009, Oldenburg, Germany)

From shorter paper provided for the trng library:
Random Number Generators: A Survival Guide for Large Scale Simulations
Stephan Mertens

Some examples are traffic simulations, economics, game simulations, physics simulations

References

Some material here is copied from these sources:

Tutorial: Random Number Generation

By John Lau Henry Yu

June 2018

http://koclab.cs.ucsb.edu/teaching/cren/project/2018/Lau+Yu.pdf

Random Number Generators: A Survival Guide for Large Scale Simulations

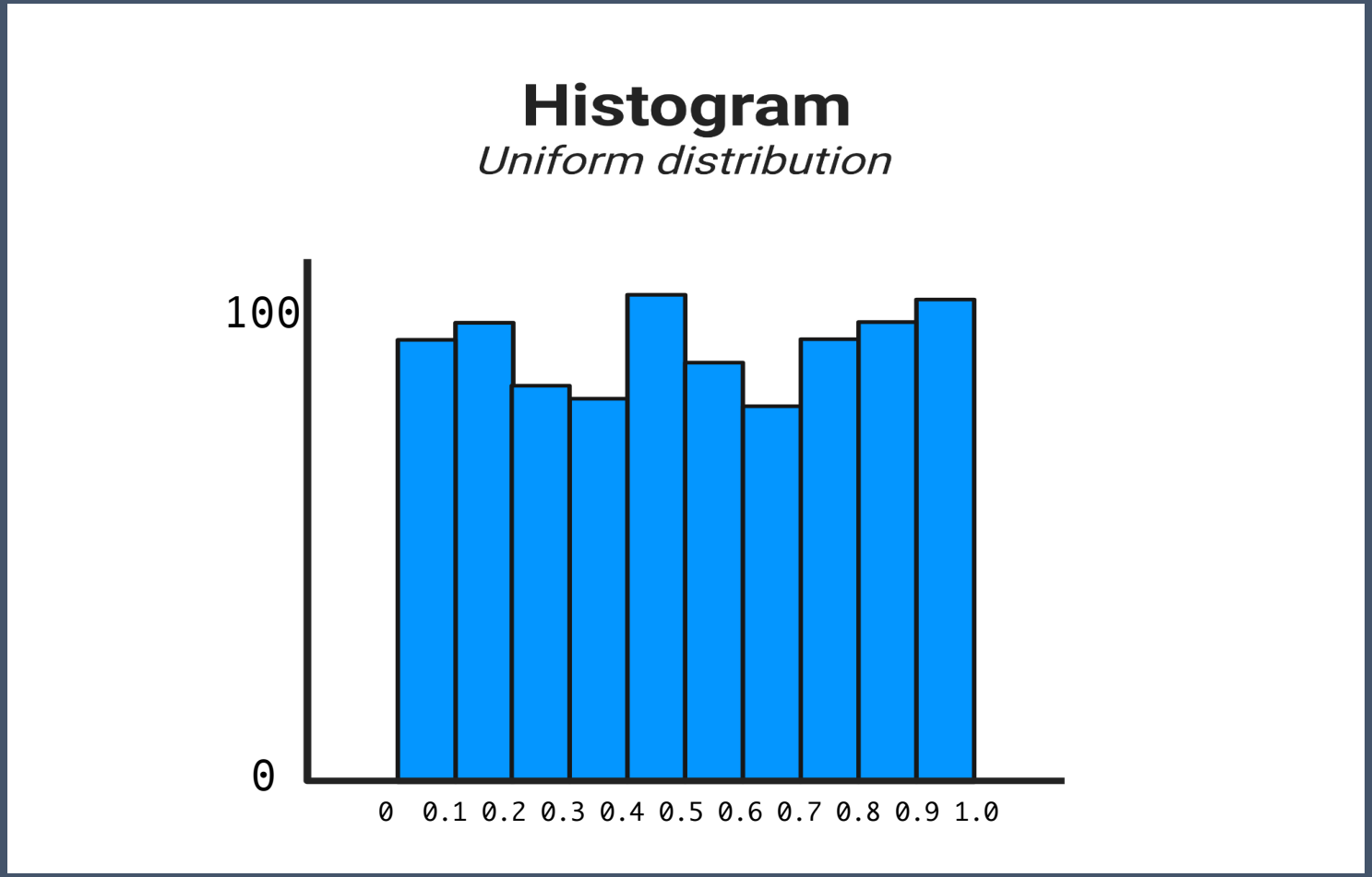
Stephan Mertens

Found as ‘shorter paper’ on our moodle site (References and OpenMP section)

Computer generated random number statistical properties

A sequence of random numbers R1, ... Ri must fulfill two statistical properties:

1. Uniformity – If the interval [0,1] is divided into n classes, or sub intervals of equal length, the expected number of observations in each interval is N/n, where N is the total number of observations



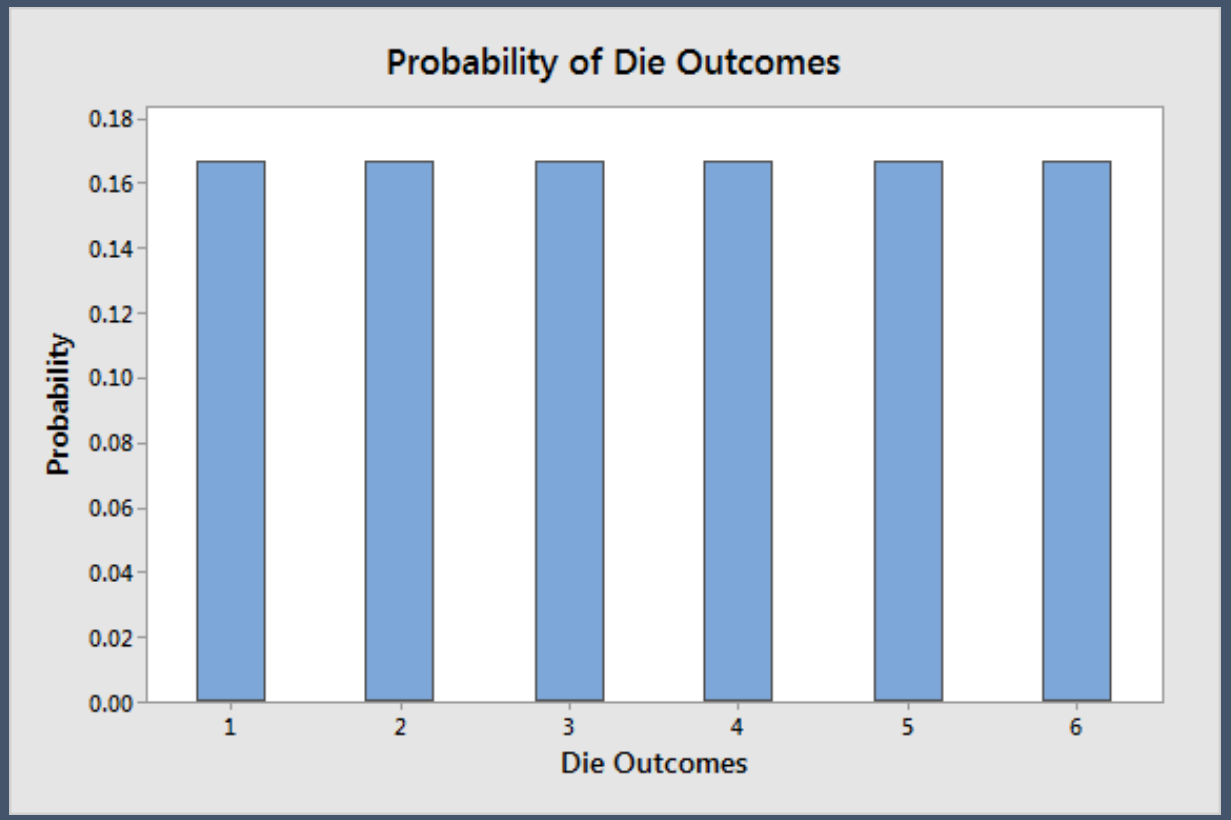
Generally computer RNGs are not perfect, but very close

If N = 1000 and n = 10
1000/10 = 100

Computer generated random number statistical properties

A sequence of random numbers R1, ... Ri must fulfill two statistical properties:

1. Uniformity – Consequence of this property: If the interval [0,1] is divided into n classes, or sub intervals of equal length, the expected number of observations in each interval is N/n, where N is the total number of observations



Perfect case shown left:
Rolling dice has six outcomes that are uniformly distributed.

Therefore, each one has a likelihood of 1/6 = 0.167. The bar chart below displays the rectangular-shaped distribution.

If we randomly generate numbers between 1 and 6, the number of times each occurs should be uniformly distributed

https://statisticsbyjim.com/probability/uniform-distribution/

Computer generated random number properties

A sequence of random numbers R1, ... Ri must fulfill two statistical properties:

2. Independence – Consequence of this property: The probability of observing a value in a particular interval is independent of the previous value drawn

Rolling a 1 is independent of rolling a 5 on the previous roll.

So too for generating numbers between 1 and 6

Other properties that random number generators should have:

- Efficiency – The generator should be fast and efficient.

- Reproducibility – The generator should be able to generate the same stream of random numbers repeatedly. This is mainly for testing and debugging purposes.

- Long Cycle Length – The generator should take a very long time before numbers start to repeat.

Popular method: Linear Congruential Method, LCM

To produce a sequence of integers, X1, X2, . . . between 0 and m-1 by following a recursive relationship

$$X_{i+1} = (aX_i + c) \bmod m$$

$$i = 0, 1, 2, \ldots$$

X_0 = is a 'seed' value, typically a large integer

Important concept: Different seeds produce different streams of random values

The selection of the values for a, c, m, and X_0 drastically affects the statistical properties and the cycle length.

An Issue with LCM

$$X_{i+1} = (aX_i + c) \bmod m$$

The numbers generated fall into planes.

If k random numbers at a time are used to plot points in k-dimensional space, then the points lie on k-1 dimensional planes rather than filling up all k-space.

There are at most m/k planes, or fewer (which is even worse) if a, c, or m are not well chosen

But modern versions have overcome this and they are quite fast, uniform, and independent

An Issue with LCM

$X_{i+1} = (aX_i + c) \bmod m$

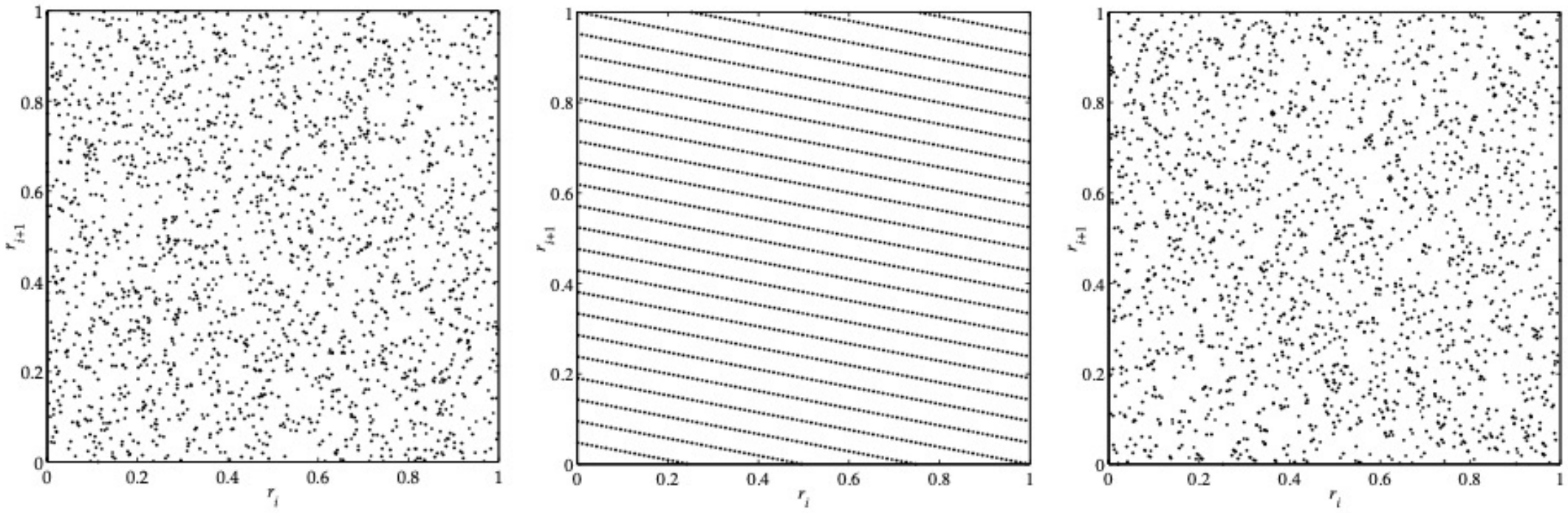


Figure 1: Points in the unit square, generated from different (pseudo)random sources.

From shorter paper provided for the trng library:
Random Number Generators: A Survival Guide for Large Scale Simulations
Stephan Mertens

A variation

linear feedback shift register sequences, or LFSR

To generate pseudo random integers between 0 and some prime number p is the linear recurrence

$$r_i = a_1 r_{i-1} + a_2 r_{i-2} + \ldots + a_n r_{i-n} \bmod p$$

Non-linear generators

To alleviate problems with linear methods,

Many non-linear generators have been tried, for example:

- “Mersenne Twister”
- Yet another random number generator (YARN)
  feed a stream of numbers from a PRNG to the Berlekamp-Massey algorithm

“Measured in terms of linear complexity, YARN generators are as random and nonlinear as it can get.”

How is this done in parallel?

We need to split the ‘stream’ of random values across the threads/processes

PRNG in parallel

In parallel applications we need to generate streams

$r_{j,i}$ of random numbers

where $j = 0,1,\ldots,p-1$ represents streams for each of the p threads/processes.

We require statistical independency of the $r_{j,i}$ within each stream and between the streams.

Efficient generation of numbers is important

You will see with examples:

- random numbers are very often needed in the innermost loop of a simulation that has nested loops

Thus, efficiency of a PRNG is very important.

We require that a PRNG can be parallelized without loosing its efficiency.

Still must be reproducible

Stephan Mertens called this ‘play fair’:

“Fair play implies the use of the same pseudo random numbers in the same context, independently of the number of parallel processes.”

What parallel methods have been tried?

1. Random Seeding

   All processes use the same PRNG but a different “random” seed. The hope is that they will generate non-overlapping and uncorrelated subsequences of the original PRNG.

   You may have done this before (or will see code that does it)

   - It’s easy to code using existing random libraries in a language

   But it turns out to be a poor way to provide one of the guarantees we want:

   - A simulation does not play fair because its results will be different depending on the number of threads/processes used

   - The set of numbers generated together by all threads does not match a sequential version with one seed

How is this done in parallel?

2. Parameterization

$X_{i+1} = (aX_i + c) \bmod m$

Use different parameters in the same type of generator, for example:

linear congruential generators with additive constant b_j for the jth stream created on each thread or process

$$r_{j,i} = a \cdot r_{j,i-1} + b_j \bmod m$$

where the b_j are different prime numbers just below m/2.

Like random seeding, parameterization prevents fair play

How is this done in parallel?

3. Block Splitting

break up a single stream of random numbers into non-overlapping, consecutive blocks of numbers and assign each block to one of the threads/processes.

Do this by:

Let M be the maximum number of calls to a PRNG by each thread/process, and

let p be the number of threads/processes.

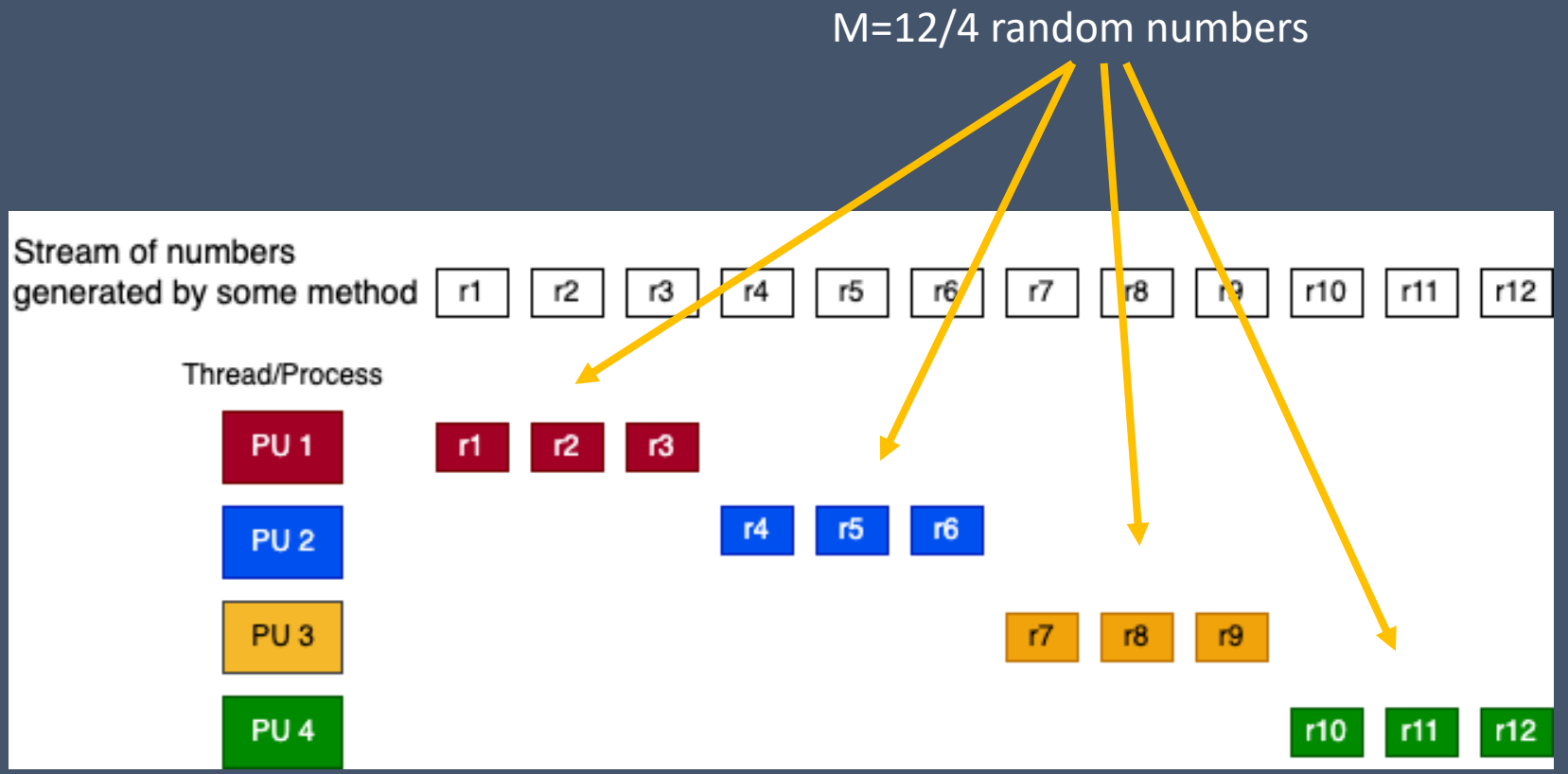
Then we can split the sequence (r) of a sequential PRNG into consecutive blocks of length M

Issues:

This method works only if we know M in advance or can at least safely estimate an upper bound for M

For an efficient block splitting, it is necessary to jump from the ith random number to the (i + p)th number without calculating the numbers in between.

Total Numbers needed should be divisible by p



This will be fair and reproducible

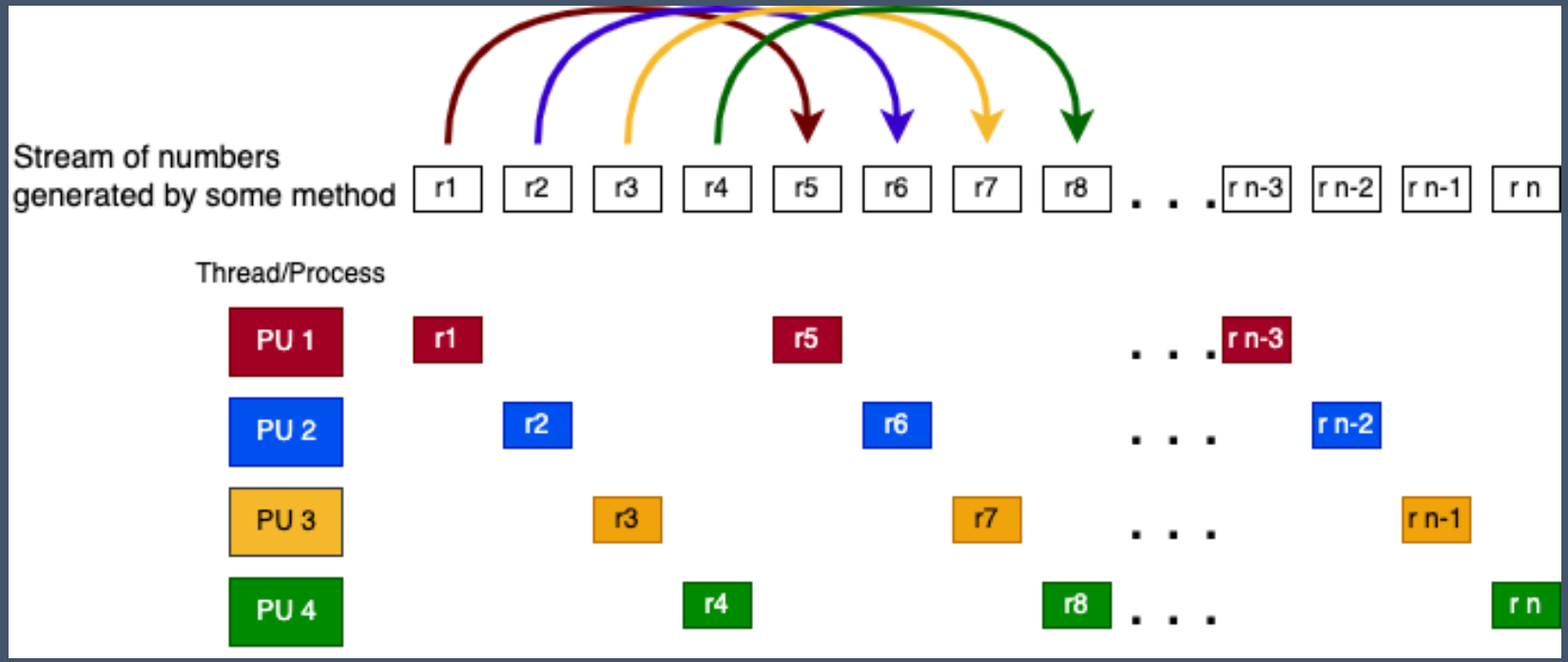
How is this done in parallel?

4. Leap Frogging

The leapfrog method distributes a sequence (r) of random numbers over d processes (shown right)

It does not require an a priori estimate of how many random numbers will be consumed by each processor.

Can in general be implemented to ensure fair play. And it reproduces the same overall set of values regardless of numbers of threads.

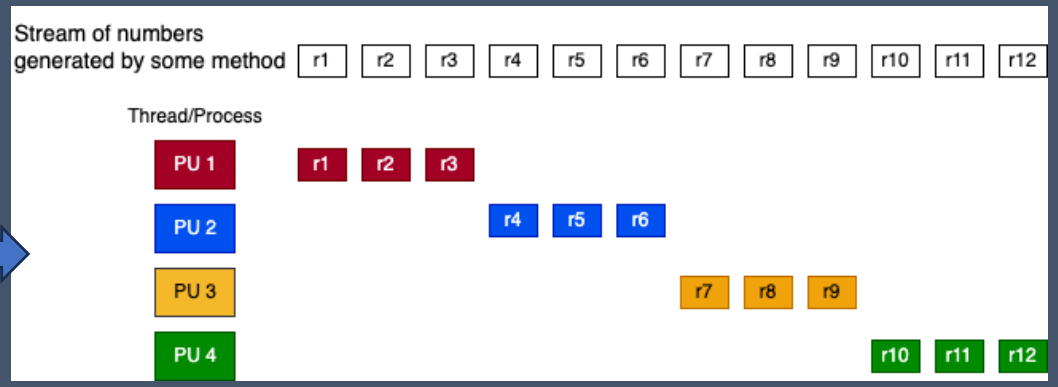


How is this done in parallel?

To do the last 2 efficiently, each thread/process will:

Directly advance the PRNG stream by M steps, then generate each number in order

for block splitting



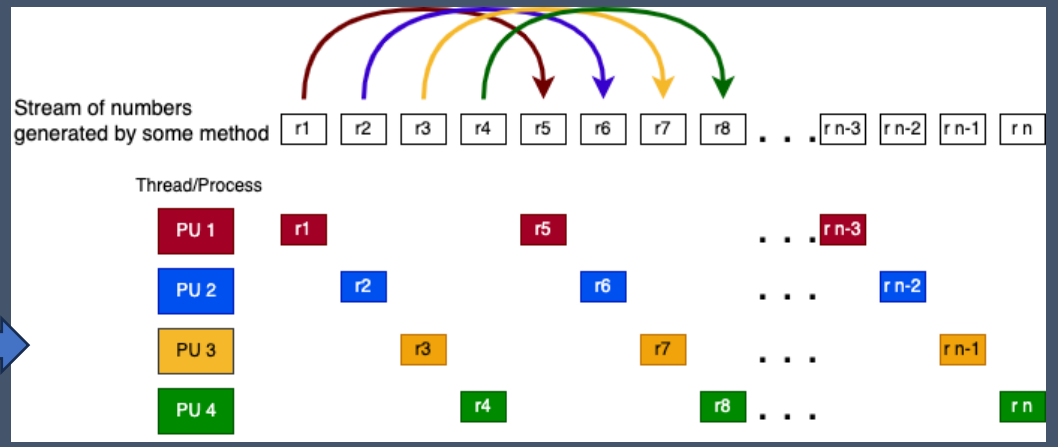
or

Modify a PRNG to generate directly only every

r_{pid} , r_{pid+p} , r_{pid+2p} , etc

element of the original sequence

for leapfrogging



Techniques for doing this have been developed, including for LSFR and YARN algorithms for generating the numbers

A library we will use: TRNG

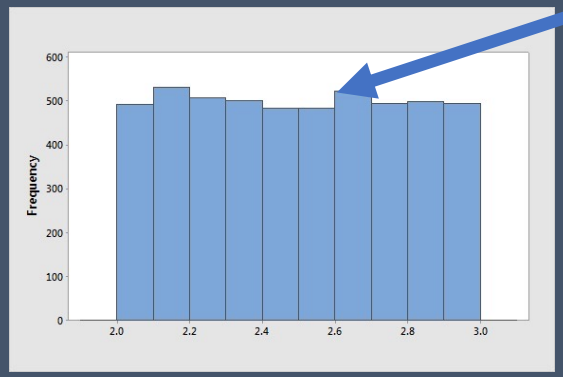
TRNG (Tina's random number generator library)

https://www.numbercrunch.de/trng/

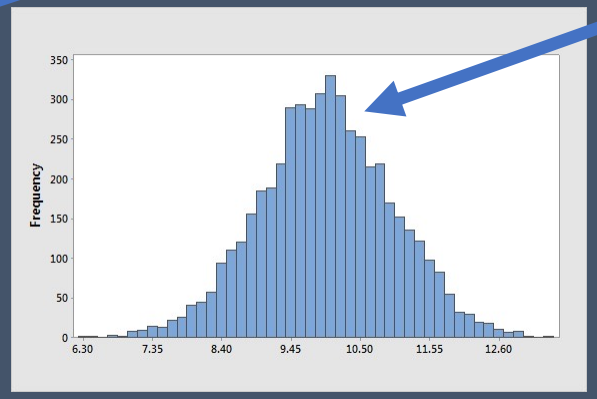
- Has several different ways of generating numbers, including LCM, LFSR and YARN
  - Using either leap-frogging or block splitting
- C++
- Has many different *distributions* of the numbers, e.g.

'bins' :
Shows how many values generated in that range of numbers

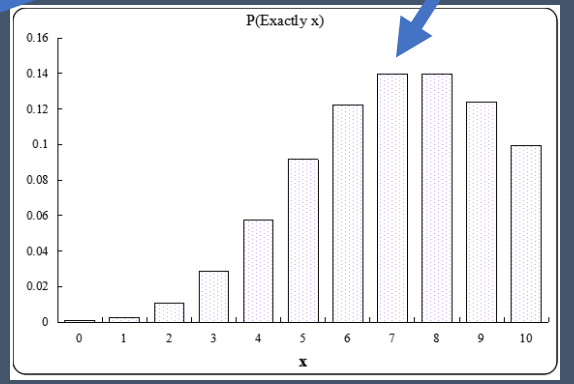
uniform



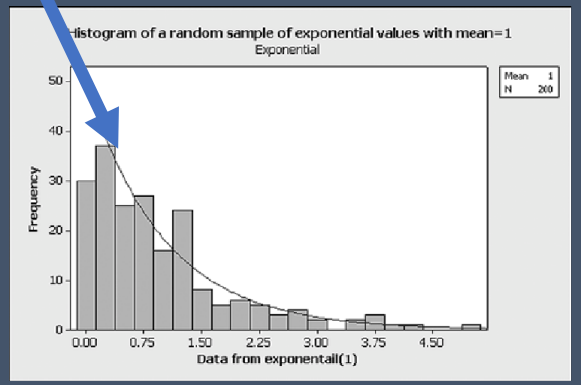
normal



poisson



exponential



Activity: practice random

Note sequential versions that use this same trng library

Note the code is C++

- study makefiles
- study code looking for C++ features